

AI-Augmented Software Development

Module 1 · The AI-Augmented Developer

Mindset, ecosystem, expectations, governance

The course thesis:

AI can accelerate software work, but developers still own the spec, architecture, review, verification, security, and long-term maintainability.

Why this course exists now

The practice changed from code suggestions to delegated engineering work

Period	Typical framing	Practical consequence
2023 - 2024	"AI helps me write code."	Autocomplete, snippets, local boilerplate
2025	"AI helps me build a feature."	Chat, multi-file edits, repo-aware work
2026	"AI can run a workflow."	Agents, tools, PRs, MCP, approvals

The new bottleneck is not typing. It is specification, context, review, verification, and safe delegation.

What “AI-augmented development” means

A precise definition prevents both hype and underuse

The intentional use of AI systems to assist, accelerate, or partially automate software work across the SDLC, while leaving accountability for correctness, safety, maintainability, and product fit with humans.

More than autocomplete

- Repo-aware explanation
- Multi-file edits
- Test and doc drafting
- PR summaries and review support

Not autonomous engineering

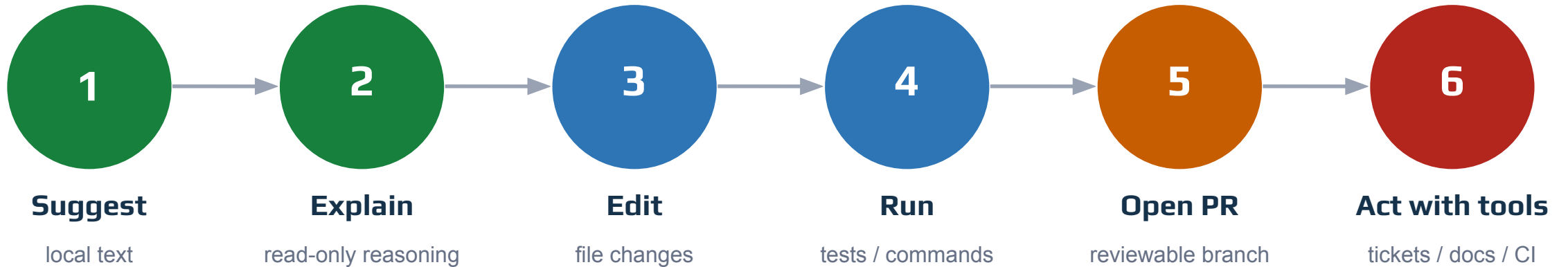
- Humans own acceptance criteria
- Humans own review and merge
- Humans own production risk

Working thesis

AI changes the mechanics of software work, not the need for engineering judgment.

From autocomplete to delegated work

The 2026 shift: more capability means more responsibility



Move right only when task clarity and verification support it.

The tool's capability is not the same as permission to use it!

The interaction spectrum

Same goal, different levels of AI responsibility



Copilot is the classroom baseline tool

One shared surface is enough to teach the assistant → agent progression

Inline assistance

Fast local completions and small edits inside the IDE.

Chat in the IDE

Explain, compare, reason, draft tests and docs with repo context.

Agentic work

Bounded edits, commands, reviews, and GitHub-side workflows where available.

Team workflow

Repository instructions, PR summaries, code review, MCP/context features.

The goal is not “learn Copilot buttons.” The goal is to learn disciplined AI-assisted engineering habits on a common tool.

Tool landscape as of June 2026

Tool / ecosystem	Useful course framing
GitHub Copilot	Default classroom surface: completions, chat, agentic work, review, repo instructions, MCP/context.
Claude Code	Repo-aware terminal/IDE agent; useful for permissions, project memory, and multi-file work.
OpenAI Codex	Agentic coding across app, IDE, CLI, and web/cloud-style workflows.
Google ecosystem	Antigravity for individual/pro agentic workflows; Gemini Code Assist still relevant for Standard/Enterprise.
Continue / OpenHands	Open, local, self-hosted, or customizable options; useful as landscape, not mandatory setup.

Product names will change. The durable skill is recognizing interaction model, autonomy level, context access, and review burden.

Assistant vs pair programmer vs. agent

Use precise language; it changes the review strategy

Assistant

You decide the next step.
AI suggests local text.
Review is mostly local diff inspection.

Pair programmer

You define the task and constraints.
AI reasons, drafts, explains, and challenges.
Review includes assumptions and tests.

Agent

You delegate a bounded outcome.
AI may inspect, edit, run, and report.
Review includes diff, commands, logs, and blast radius.

Bad shortcut: “The AI did it.”

Better framing: “I delegated this bounded task, reviewed the diff, ran these checks, and accepted this risk.”

Use a five-question decision model

Choose the mode before writing the prompt

- 1 **How clear is the task?**
- 2 **How much context is required?**
- 3 **What is the blast radius if it is wrong?**
- 4 **Can the result be verified cheaply?**
- 5 **Do you need suggestions, or do you need the tool to act?**

Rule of thumb: increase autonomy only when clarity and verification improve faster than risk.

Tool-fit examples

AI use is not binary; the mode should match the task

Task	Recommended mode	Why
Rename a local variable	Assistant	Low risk, local diff
Explain a legacy parser	Chat / pair programmer	Read-only orientation
Generate tests for pure function	Pair programmer	Useful draft; assertions need review
Implement small isolated issue	IDE agent	Bounded and testable
Modify auth, billing, migrations	Human-led AI support only	High blast radius
Summarize bug thread	Chat assistant	Useful draft; verify facts
Multi-step repo task	Coding agent with PR review	Clear issue + reviewable diff

Where current AI tools are usually strong

These strengths are real — but they are not guarantees.

Scaffolding and boilerplate

Explaining unfamiliar code

Repetitive transformations

Drafting tests and docs

Refactoring familiar patterns

Repo orientation and search

Structured work under constraints

Useful first drafts quickly

Why it feels magical on day one: models are good at turning bounded developer intent into a plausible first draft.

The hidden failure mode: plausible incompleteness

The output often looks finished before it is safe to own

- ▶ Hidden business rules and edge cases
- ▶ Security-sensitive defaults
- ▶ Dependency and version assumptions
- ▶ Confident but wrong explanations
- ▶ Tests that pass while checking the wrong thing
- ▶ Code that looks finished but is only 80% correct

Look for what is missing, not only what is present.

Generated prose can be smoother than the implementation is sound.

Read the diff before trusting the explanation.

Mini-case: the confident intern problem

A polished answer can still be unsafe.

AI answer excerpt

“The parser is mostly production-ready. Log the full incoming payload, add this example token to the README, use `safeprofile.validate_everything()`, let the agent update any files, and run the happy-path test once.”

Student task

Find issues under: privacy, invented behavior, security risk, missing verification, maintainability risk, over-delegation.

Teaching point

The answer is dangerous because it is smooth enough to sound reasonable.

Debrief line

Governance is not paperwork. It is how we prevent plausible mistakes from entering the codebase.

Privacy, IP, and governance guardrails

Safe AI use starts with data boundaries and review ownership

Usually safe

- Toy code
- Synthetic examples
- Public docs
- Approved summaries
- Redacted stack traces

Share with caution

- Proprietary code
- Architecture details
- Logs and stack traces
- Security-sensitive material
- Non-public product plans

Never paste

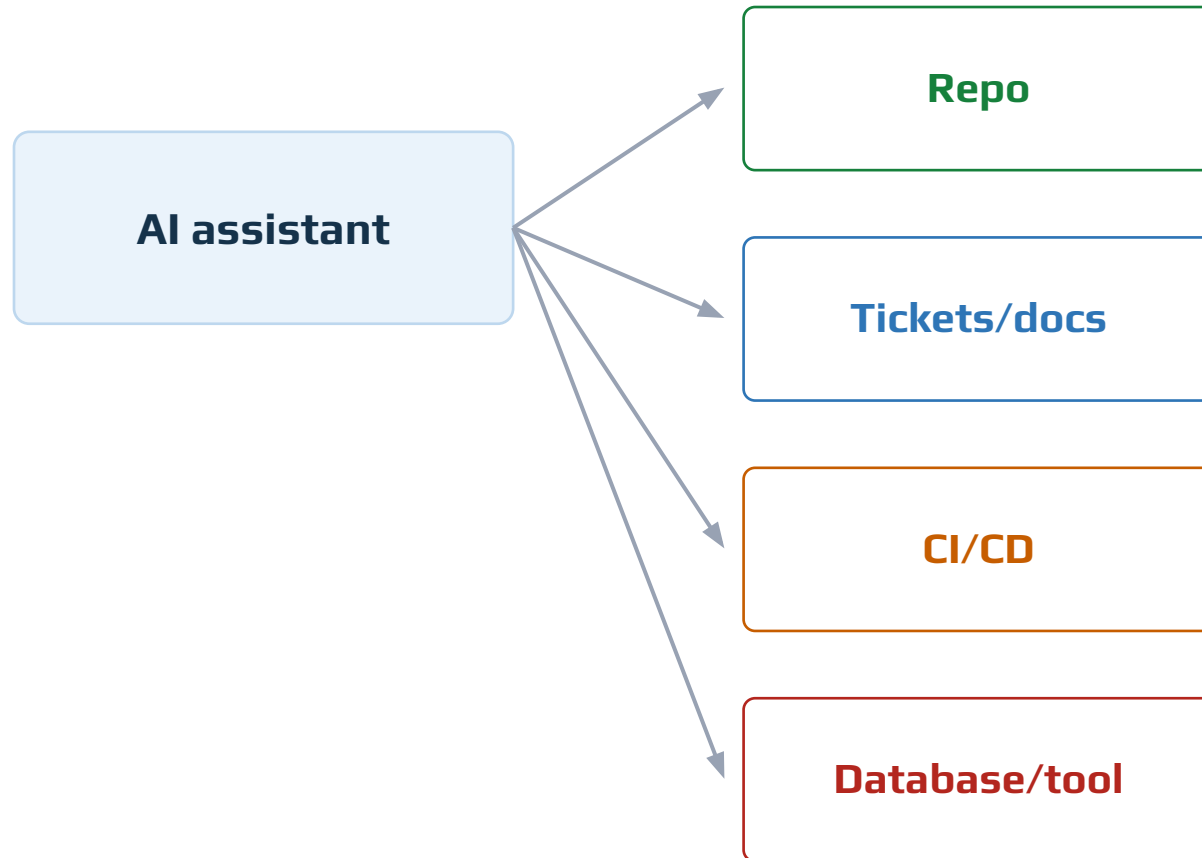
- Secrets, keys, tokens
- Live customer data
- Regulated personal data
- Production dumps
- Sensitive payloads

Human ownership stays with acceptance criteria, review, verification, and merge decisions.

Optional preview: connected tools and MCP

OPTIONAL

Tools extend the assistant — and the risk surface.



Preview takeaway

Once tools can read from and act on external systems, governance stops being optional.

In-class exercise: classify the AI mode

For each task: mode, safe context, verification plan.

Task	Mode?	Safe context?	Verification?
Generate Pydantic models from JSON samples			
Explain a legacy parser before changing it			
Add tests to a pure utility package			
Refactor auth middleware			
Fix a flaky test			
Implement payment retry logic			

Focus on reasoning, not on finding a single official answer.

Key takeaways

The mental model for the rest of the course

- ▶ Think in interaction modes, not brand names.
- ▶ Increase autonomy only when the task is clear and reviewable.
- ▶ Expect acceleration, not guaranteed correctness.
- ▶ Treat generated code as untrusted until reviewed.
- ▶ Keep privacy, IP, and governance in the workflow from day one.
- ▶ Context is repository structure, tests, docs, conventions, and constraints — not just prompt text.

Next: disciplined AI-assisted development inside a normal engineering workflow.